

# Concurrency and Distributed Systems

## Week 5 – Part 5: ThreadLocal Request Context and Checkpoint 2

Dr. Mohamad Aoude

Faculty of Engineering

May 11, 2026

# Part 5 Roadmap

- 1 Why Request Context Exists
- 2 ThreadLocal Semantics
- 3 Leaks in Thread Pools
- 4 Safe Cleanup Patterns
- 5 Exercise 5
- 6 Checkpoint 2 Integration

# Why This Part Matters

## Core question

How do we associate request-specific data with the current worker without passing it through every method?

In server code, many helpers need request context:

- request ID,
- user ID,
- tenant or group,
- trace/correlation ID,
- request deadline.

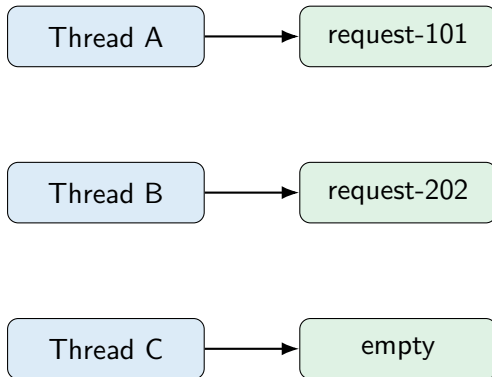
# The Context Passing Problem

- Passing context explicitly is clear, but can be verbose.
- Global variables are unsafe because requests run concurrently.
- Static mutable fields mix all requests together.
- Thread-local storage gives each worker thread its own value.

## Warning

Thread-local storage is scoped to a thread, not to a request.

# ThreadLocal Mental Model



## Key idea

One `ThreadLocal` variable stores separate values per thread.

# ThreadLocal API

```
1 ThreadLocal<String> requestId = new ThreadLocal<>();  
2  
3 requestId.set("req-123");  
4 String current = requestId.get();  
5 requestId.remove();
```

- set: store value for the current thread.
- get: read current thread's value.
- remove: clear current thread's value.
- withInitial: create default values lazily.

## Demo12: Request Context

```
1 private static final ThreadLocal<String> REQUEST_ID =  
2     new ThreadLocal<>();  
3  
4 public static String runWithRequestId(String requestId) {  
5     REQUEST_ID.set(requestId);  
6     try {  
7         return REQUEST_ID.get();  
8     } finally {  
9         REQUEST_ID.remove();  
10    }  
11 }
```

### What students should notice

The demo sets context, uses it, and removes it in the same control path.

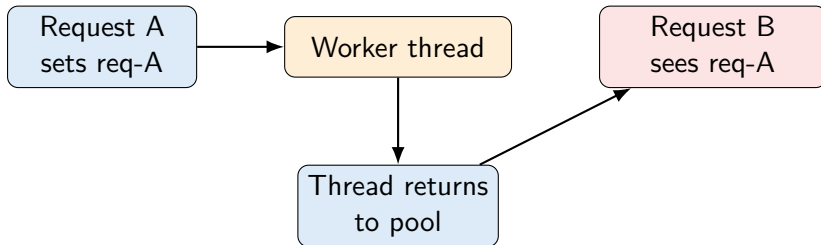
# Why Cleanup Is Mandatory

- In a thread-per-request model, the thread may die after the request.
- In a pool, the thread survives and handles later requests.
- A stale `ThreadLocal` value can become visible to the next task on that worker.
- This can leak identity, trace IDs, permissions, or tenant data.

## Rule

If you set a `ThreadLocal`, you must know exactly where it is removed.

# Thread Pool Reuse



## The bug

The request ended, but the worker thread kept the value.

## Demo13: Unsafe Handler

```
1 public static String unsafeHandle(String requestId, boolean setValue) {  
2     if (setValue) {  
3         REQUEST_ID.set(requestId);  
4     }  
5     return REQUEST_ID.get();  
6 }
```

- First call sets a value.
- Later call may skip set.
- Same worker still has the old value.
- There is no automatic cleanup.

# What Makes This Bug Dangerous

- It depends on worker reuse, so it may be intermittent.
- It may not appear in single-threaded unit tests.
- It can leak data across requests.
- Logs and metrics may be attributed to the wrong request.
- Authorization or tenancy mistakes can become severe.

## Safe Pattern: try/finally

```
1 REQUEST_ID.set(requestId);  
2 try {  
3     handleRequest();  
4 } finally {  
5     REQUEST_ID.remove();  
6 }
```

### Rule

The `finally` block must run whether the request succeeds, fails, or throws.

# Safe Wrapper Method

```
1 public static void runWithContext(String requestId, Runnable task) {  
2     REQUEST_ID.set(requestId);  
3     try {  
4         task.run();  
5     } finally {  
6         REQUEST_ID.remove();  
7     }  
8 }
```

- Centralizes the lifecycle.
- Reduces caller mistakes.
- Makes tests simpler.
- Mirrors how request filters/interceptors often work.

# Ownership Rule

## Do not scatter set/remove

If one method sets context and another method is expected to clear it, the design is fragile.

Prefer:

- one wrapper method,
- one filter/interceptor,
- one try/finally boundary,
- tests proving cleanup.

# Testing ThreadLocal Cleanup

- Run two tasks on a single-thread executor.
- First task sets context.
- Wrapper must clear after the first task.
- Second task must observe no stale context.

## Why single-thread executor?

It forces worker reuse, making the leak deterministic.

## Exercise 5: RequestContext

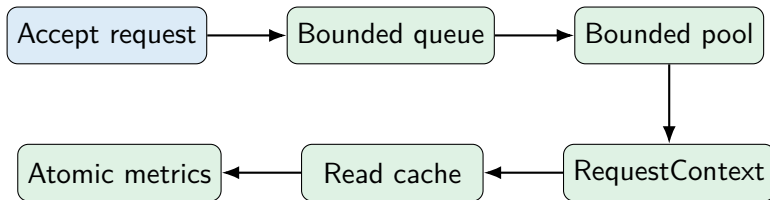
```
1 public final class Ex5_RequestContext {  
2     public static void set(String requestId) { ... }  
3     public static String get() { ... }  
4     public static void clear() { ... }  
5     public static void runWithContext(String requestId, Runnable task) { ... }  
6 }
```

- Store request ID for the current thread only.
- Clear current thread context.
- Provide safe wrapper execution.
- Prove no leak across reused pool workers.

## Common Mistakes in Ex5

- Using a static `String` instead of `ThreadLocal<String>`.
- Forgetting `remove()`.
- Calling `task.run()` before setting context.
- Clearing only when the task succeeds.
- Catching exceptions and hiding failure from the caller.

## Checkpoint 2 Server Flow



## Checkpoint 2 Mapping

| Exercise | Server responsibility            |
|----------|----------------------------------|
| Ex1      | Bounded request queue.           |
| Ex2      | Bounded worker execution.        |
| Ex3      | Atomic success/failure metrics.  |
| Ex4      | Read-heavy cache/config lookup.  |
| Ex5      | Per-request context and cleanup. |

# Checkpoint 2 Evidence

Students should be ready to show:

- passing tests,
- no unbounded queue in the request path,
- no thread-per-request design,
- rejection or backpressure behavior,
- atomic metrics under concurrent updates,
- no ThreadLocal leak after task completion.

# Scorecard Notes

- Capacity: what is bounded?
- Liveness: what can block, and what unblocks it?
- Fairness: can one request starve others?
- Visibility: what shared data needs safe publication?
- Cleanup: what is released at the end of each request?

# Week 5 Final Takeaway

## Bounded concurrency

Every queue, pool, cache, counter, and context value should have a clear owner, capacity, lifecycle, and cleanup rule.

## Bridge to Week 6

Now that execution is controlled, we can ask how to split computation itself:

parallelism and Fork/Join.

# Questions?